COMPONENT BASED SOFTWARE ENGINEERING

Component Engineering Process

DEFINITION

- The component engineering process involves a series of activities used to identify, analyze, specify, develop, document, test, store, deploy, and maintain software components for building software systems.

1. Component identification

- Component identification is the process of determining which software components are required to satisfy the system's requirements.
- Activities:
  - Analyze system requirements.
  - Identify the functions the system must perform.
  - Search for existing reusable components.
  - Determine which components need to be developed.
- Example: In a mobile banking application, identifying login, account management, and money transfer components.

2. Component analysis

- Component analysis is the process of examining identified or available components to determine whether they are suitable for use in the intended system.
- Activities:
  - Evaluate component functionality.
  - Check compatibility with the system architecture.
  - Assess reliability, security, performance, and reusability.
  - Examine dependencies, licensing, and limitations.
- Example: Checking whether an existing authentication component supports the security requirements of a mobile banking application.

3. Component specification

- Component specification is the process of defining the requirements, responsibilities, interfaces, and expected behaviour of a software component.
- A component specification describes what a component should do and how other components can interact with it.
- Activities:
  - Define component functions.
  - Specify inputs and outputs.
  - Define interfaces and dependencies.
  - Establish performance and security requirements.
- Example: Specifying that a login component accepts a username and password and returns an authentication result.

4. Component development

- Component development is the process of designing, coding, and implementing a software component according to its specification.
- Activities:
- Select appropriate programming languages and tools.
- Design the component's internal structure.
- Write and compile the code.
- Implement required interfaces.
- Follow coding standards and best practices.
- Example: Programming a transaction component that transfers money between bank accounts.

5. Component documentation

- Component documentation is the process of preparing information that explains a component's purpose, functionality, interfaces, usage, and limitations.
- Typical documentation includes:
- Component description and purpose.
- Installation and configuration instructions.
- Interface and API details.
- Usage examples.
- Dependencies and version information.
- Known limitations and troubleshooting guidance.
- Example: Documenting how developers can call a payment component's API and interpret its responses.

6. Component testing

- Component testing is the process of verifying that an individual software component operates according to its specification.
- Types of testing may include:
- Unit testing: Checks individual functions or units within a component.
- Functional testing: Verifies that the component performs its intended tasks.
- Interface testing: Checks communication through the component's interfaces.
- Security testing: Checks for vulnerabilities.
- Performance testing: Evaluates response time and resource usage.
- Example: Testing whether a payment component correctly processes valid payments and rejects invalid requests.

7. Component repositories

- A component repository is a centralized or distributed storage facility where reusable software components and their related information are stored, organized, and retrieved. They function as centralized or distributed databases where verified software components are stored, indexed, and made accessible to development teams
- A repository may contain:
  - Component source code or compiled packages.
  - Documentation.
  - Version information.
  - Dependencies.
  - Interface specifications.
  - Testing results and metadata.
- Examples: GitHub repositories, Maven Central for Java packages, and npm for JavaScript packages.

Cont.: Importance:

- Promotes component reuse.
- Makes components easier to locate and retrieve.
- Supports version management.
- Reduces duplication of development effort.

8. Component deployment

- Component deployment is the process of installing, configuring, and making a software component available in its intended operating environment.
- Activities:
  - Package the component.
  - Configure dependencies and settings.
  - Install it in the target environment.
  - Integrate it with other components.
  - Verify its operation after deployment.
- Example: Deploying a payment service to a production server so that a mobile banking application can use it.

9. Component maintenance

- Meaning: Component maintenance involves modifying, updating, repairing, and improving software components after they have been developed or deployed.
- Types of maintenance include:
  - Corrective maintenance: Fixes defects and errors.
  - Adaptive maintenance: Adjusts a component to changes in its operating environment.
  - Perfective maintenance: Improves performance, usability, or functionality.
  - Preventive maintenance: Reduces the likelihood of future problems.
- Example: Updating an authentication component to address a security vulnerability or support a new authentication method.

End

!!!!!!!